Scalable Fine-Grained eBPF Isolation through Hybrid Policy Verification

Abstract: Multi-tenant cloud hosts run eBPF programs for networking, observability, and security on a shared Linux kernel. However, kernel-verifier acceptance does not guarantee tenant-specific policy compliance. This paper presents a selective-analysis architecture using a conservative three-valued abstract analyzer (SAFE, VIOLATION, UNKNOWN), with unresolved cases forwarded to a frozen KRAKENGUARD symbolic verifier. Baselines E1 (8 programs, 72 runs) and E2 (1–64 feasible paths, 63 runs) show that instruction count does not drive cost (672–708 ms), whereas path count scales near-linearly (746–1,231 ms, ~7.7 ms/path above ~730 ms fixed overhead). Comparative workbench evaluation of the 24-program corpus across 504 measured runs demonstrates that the abstract stage discharged 50.0% (12/24) of programs without fallback, achieving 46.16% paired median wall-time savings with zero false acceptances or rejections across 420 definitive checked runs (and 84 conservative UNKNOWN abstentions). However, overall speedup across the balanced corpus is not statistically significant (two-sided sign test p = 1.0; bootstrap 95% CI: -31.62 to +670.72 ms). Furthermore, evaluation on Meta's production Katran load balancer compiled successfully to ELF bytecode (134 KB), but failed during bytecode lifting due to an LLVM opaque pointer assertion crash in KRAKENGUARD. This study establishes empirical safety on synthetic workloads, quantifies hybrid verification speedup trade-offs, and identifies front-end lifter robustness as the primary barrier to production deployment.

Keywords *eBPF, multi-tenant cloud security, eBPF isolation, symbolic execution, abstract analysis, policy verification, KRAKENGUARD*

1. Introduction

1.1 Background of the Study

Cloud infrastructure increasingly consolidates workloads on shared Linux hosts. Extended Berkeley Packet Filter (eBPF) supports networking, tracing, observability, and security [1], but allows programs from different tenants to run within the same kernel. Security mechanisms must distinguish each tenant’s authorized actions. Although the Linux verifier provides a fundamental safety check, verifier acceptance does not guarantee tenant-specific policy compliance: a program may pass verification yet violate policies governing helper calls, maps, memory, return values, or interactions with other programs.

1.2 Research Problem

Fine-grained symbolic policy analysis, as used in KRAKENGUARD [2], can precisely evaluate policy compliance but incurs the full verification cost for every program. Its scalability is limited by growing symbolic paths and expressions. Running symbolic verification on every program is unnecessary when compiled structure makes the decision clear. This study therefore seeks to reduce redundant symbolic verification without treating uncertainty as safe.

1.3 Significance of the Study

A fail-closed selective front-end preserves the symbolic verifier as the authority while avoiding its cost when a cheap proof is available. Baseline measurements identify whether program size, path count, or fixed overhead drives verification time. The study also distinguishes measured findings from planned or unrecoverable results, supporting more transparent systems-security evaluations.

1.4 Research Gap

Existing research addresses verifier precision [3], runtime enforcement [4], verifier testing [5], virtualization [6], scheduling [7], delegation [8], and confinement [14], [15]. However, limited work examines whether a conservative three-valued abstract stage can safely reduce symbolic policy-verification calls while reporting audited cost measurements across program features and feasible paths. This study addresses that gap through empirical evaluation of selective discharge, path scaling, and real-world deployment feasibility.

1.5 Objectives and Research Questions

The objective is to design a conservative hybrid policy-verification workflow, measure the baseline cost behavior of the symbolic verifier, and state exactly what evidence supports each claim. The study addresses four research questions:

1. **RQ1:** Does compiled instruction count explain end-to-end symbolic verification cost (E1)?
2. **RQ2:** How does cost grow with the number of feasible symbolic paths (E2)?
3. **RQ3:** Can a conservative three-valued analyzer discharge a useful subset of programs without false SAFE or false VIOLATION decisions (E3)?
4. **RQ4:** Under what conditions does the hybrid workflow reduce end-to-end cost, and how does fallback behave as path complexity increases (E4, E5)?

1.6 Scope of the Study

The study evaluates synthetic XDP programs compiled with Clang for BPF, using one fixed policy, KRAKENGUARD revision, and host. It neither replaces the Linux verifier or KRAKENGUARD nor claims end-to-end tenant isolation. Its contributions are:

1. A characterization of KRAKENGUARD through controlled program-feature (E1) and symbolic-path (E2) experiments.
2. A conservative abstract policy analyzer with SAFE, VIOLATION, and UNKNOWN outcomes and fail-closed fallback.
3. A cost model and break-even analysis of selective verification.
4. An empirical audit evaluating hybrid verification trade-offs across 504 measured runs, reconciling historical Phase 5 claims, and identifying real-world bytecode lifting boundaries.

2. Review of Literature

2.1 eBPF in Multi-Tenant Infrastructure

eBPF allows user-supplied programs to run at selected kernel hooks; XDP, for example, enables fast packet processing near the receive path. The kernel documents instruction behavior, helpers, maps, and verifier concepts [10], while RFC 9669 standardizes the BPF instruction set [11]. Runtime surveys describe how in-kernel execution, helpers, and verification interact [1]. This programmability also creates an isolation requirement: each tenant’s program must remain within its authorized scope.

2.2 Abstract and Symbolic Analysis

Abstract interpretation summarizes program behavior in a tractable domain, conservatively sacrificing precision when needed [12]. Symbolic execution explores path conditions and solves constraints; KLEE is a well-known engine for LLVM programs [13]. KRAKENGUARD lifts eBPF bytecode into an intermediate representation, resolves helpers and symbols, builds a verification harness, and uses KLEE to examine policy-relevant behavior [2]. Because symbolic execution may explore many

feasible paths, its cost depends on workload characteristics—a relationship this paper measures rather than assumes.

2.3 Related eBPF Isolation and Verification Systems

Table I summarizes related work. These systems motivate the multi-tenant context but are not interchangeable baselines; this study compares a hybrid strategy with a frozen KRAKENGUARD configuration.

Table I: Summary of related eBPF isolation and verification work

Work	**Focus**	**Main technique**
KRAKENGUARD [2]	Policy verification	KLEE-based fine-grained isolation policies
BCF [3]	Verifier precision	Proof-guided abstraction refinement
AEE [4]	Runtime enforcement	Approximation-enforced execution of untrusted extensions
Veritas [5]	Verifier testing	Specification-based oracle for misbehavior
vBPF [6]	Virtualization	Tenant and state isolation with late binding
PeeR [7]	Scheduling	Isolation for latency-critical eBPF applications
BPF Token [8]	Delegation	Namespace-bound BPF filesystems
SeaBee [9]	Infra. protection	Defense against privileged tampering
bpfbox [14]	Confinement	Process confinement with eBPF policy
BPFContain [15]	Containers	eBPF-based container confinement
This study	Selective verification	Three-valued abstract front-end with symbolic fallback

2.4 Confinement and Isolation Risks

bpfbox demonstrates precise process confinement through eBPF-based policy enforcement [14], while BPFContain examines container confinement [15]. Research on cross-container attacks shows how overly permissive eBPF capabilities threaten isolation in shared-kernel clouds [16]. These studies inform policy design and threat modeling but do not provide performance baselines for policy verification.

2.5 Need for the Present Study

Generic multi-tenant isolation is already addressed by several systems and is not this study's novelty. The narrower, falsifiable question is whether a conservative front-end can reduce symbolic invocations without changing final decisions, and how symbolic verification costs scale with program size and path count. This study provides baseline measurements and an evidence-bounded design.

3. Research Methodology

3.1 Research Design and Approach

The study uses a five-phase design-and-evaluate approach: literature taxonomy, gap verification, baseline measurement, controlled experiments (E1 and E2), and hybrid prototyping with comparative

workbench evaluation. Analysis is descriptive and inferential. All reported metrics are verified against raw execution records.

3.2 System Architecture

Figure 1 shows the hybrid workflow. Analysis uses the compiled eBPF object and control-flow graph, not C source, because compilation may fold predicates or remove branches [10], [11].

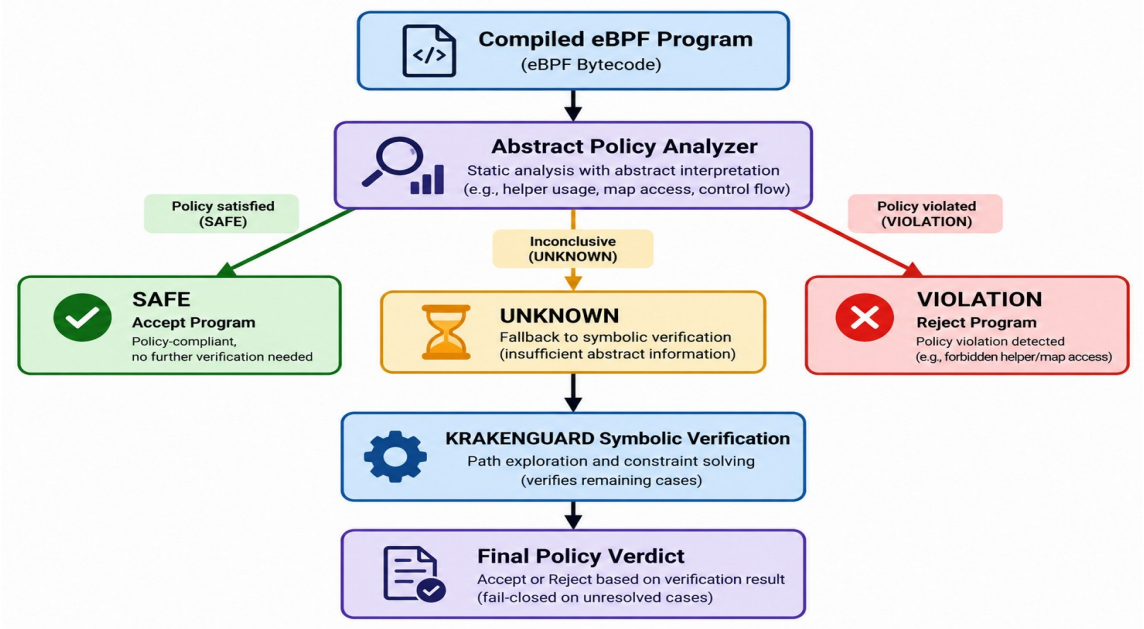


Figure 1. Hybrid fine-grained eBPF policy-verification pipeline. The abstract analyzer accepts or rejects only on a conclusive result; unresolved cases go to KRAKENGUARD, and errors or timeouts never imply acceptance.

1) Input and compilation. Programs are compiled with Clang for BPF (-mcpu=v1, -O2 -g, XDP hook). Source and object hashes, policy hash, and compiler settings are recorded to identify each tested binary.

2) Abstract policy analysis. The abstract state tracks reachability, integer facts, policy predicates, helper and map identities, access modes, memory facts, return values, and analysis status. Facts propagate through the control-flow graph; branches are pruned only when predicates are proven, and conflicting facts are weakened at joins. Unsupported eBPF semantics are never assumed compliant.

3) Three-valued routing. The analyzer returns one outcome: SAFE when all reachable abstract states are proven compliant, VIOLATION when a forbidden action is proven reachable, or UNKNOWN when neither is established. A forbidden operation's syntactic presence alone does not establish VIOLATION; reachability must be proven. With A the abstract analyzer and V_{sym} the authoritative symbolic verifier, the hybrid decision is:

$$H(P,\Pi) = \text{ACCEPT}, \quad \text{if } A(P,\Pi) = \text{SAFE} \tag{1}$$
$$H(P,\Pi) = \text{REJECT}, \quad \text{if } A(P,\Pi) = \text{VIOLATION}$$
$$H(P,\Pi) = V_{sym}(P,\Pi), \quad \text{if } A(P,\Pi) = \text{UNKNOWN}$$

The design is one-sided: analysis failure reduces precision, never authority; UNKNOWN never implies SAFE. Table II summarizes the required actions.

Table II: Hybrid decision semantics and required action

Abstract result	Evidence available	Required action
SAFE	Compliance proved	Accept
VIOLATION	Violation proved	Reject
UNKNOWN	Neither proof	Symbolic fallback
Analysis error	No trusted result	Fail closed
Timeout	No completed proof	Do not accept

4) Symbolic fallback. Unresolved programs enter the frozen KRAKENGUARD pipeline, which lifts bytecode, builds a harness, and runs KLEE [2], [13]. Only the authoritative conditional-policy result determines the verdict; transport success, a top-level response flag, or process exit without a valid policy artifact is insufficient.

5) Verdict and provenance. Violations are rejected, while timeouts, malformed responses, missing verdicts, and analysis errors remain distinct and never count as success. Each run records the abstract decision, uncertainty reason, fallback status and verdict, per-stage timing, and artifact identifiers.

3.3 Study Area and Context :

All experiments ran on one host with a frozen KRAKENGUARD revision (e7bd840…) and a fixed policy (SHA-256 270403…), packaged in a Docker artifact with a pinned image digest (Table III).

Table III: Experimental environment

Item	Value
Operating system	Linux 7.2.3-arch1-2, x86_64
CPU / memory	AMD Ryzen 5 5600H, 12 logical cores; about 7.1 GiB RAM
Toolchain	Clang/LLVM 22.1.8; -target bpf -mcpu=v1 -O2 -g
Hook type	XDP
Baseline	Frozen KRAKENGUARD, Docker image (immutable digest)

3.4 Population, Sample and Sampling Technique

The target population is policy-specific eBPF programs on shared hosts. Since production corpora were unavailable, synthetic programs were purposively constructed. E1 uses eight XDP micro-programs (2–20 instructions), while E2 tests 1, 2, 4, 8, 16, 32, and 64 feasible KLEE paths. The Phase 5 corpus contains 24 programs across four equal categories (Table IV), distinguishing easy cases from those requiring fallback.

Table IV: Validation corpus (24 programs)

Cat.	Description	Programs
A	Abstractly provable compliant	6
B	Abstractly uncertain, symbolically compliant	6
C	Abstractly provable violation	6
D	Abstractly uncertain, symbolically violating	6

3.5 Data Sources, Collection Methods and Research Instruments

The primary endpoint is end-to-end wall time. Each program or level has two warmups and seven measured repetitions: E1 includes 72 executions (56 measured) and E2 63 (49 measured). The comparative evaluation comprises 648 total runs (504 measured across 24 programs and 3 execution modes) interleaved under seed 42. E5 evaluates path scaling across 1–32 feasible paths. Phase 5 hypotheses evaluate agreement with the symbolic reference (H5-C), selective discharge (H5-COV), hybrid wall-time savings (H5-P), and path-dependent fallback cost (H5-S).

3.6 Data Analysis Techniques

Medians and means are reported per program or level; outliers are retained unless a predeclared rule invalidates them. For path scaling, a least-squares line is fitted to the medians and described as an empirical model, not a complexity result. For the hybrid workflow let A_i, S_i and F_i be abstract time, symbolic-only time and incremental fallback time for program i, and let $I_i = 1$ if the result is UNKNOWN and 0 otherwise:

$$H_i = A_i + I_i F_i, \quad \Delta_i = H_i - S_i \tag{2}$$

A negative Δ_i favors the hybrid. With discharge fraction d and mean costs, the expected hybrid cost and the break-even condition are:

$$E[H] = \bar{A} + (1 - d)\bar{F} \tag{3}$$

$$\bar{A} + (1 - d)\bar{F} < \bar{S} \tag{4}$$

For E4 the unit of comparison is the program: with $H\bar{e}_i$ and $S\bar{e}_i$ the per-program medians, $d_i = H\bar{e}_i - S\bar{e}_i$ and $r_i = H\bar{e}_i / S\bar{e}_i$. A ratio of aggregate medians is not the median of per-program ratios, so the denominator and aggregation order must be stated.

3.7 Ethical Considerations

The study involved no human participants or personal data. All programs were synthetic, and open-source tools were used under their respective licenses. The findings represent an empirical research evaluation, not a production security guarantee. All reported experimental metrics are derived directly from verified raw execution records.

4. Results

4.1 E1: Program Features and Verification Cost

The E1 audit confirmed 72 rows, unique run IDs, consistent policy and baseline IDs, and exact agreement between CSV measurements and JSON aggregates, with qualifications. Table V and Figure 3 show median times of 672.4–707.6 ms (a 35.2 ms, or 5.2%, spread) despite a tenfold increase in compiled instructions. The correlation between instruction count and median time was (r=0.41) across eight programs, driven by P7, the only two-path program; among the seven single-path programs, (r\approx-0.05). P4’s genuine timing spike was retained.

Table V: E1 median end-to-end verification time

Program	Instructions	Paths	Median (ms)
P0	2	1	681.82
P1	5	1	679.87
P2	4	1	677.55
P3	18	1	672.43
P4	6	1	688.57
P5	11	1	692.90
P6	17	1	686.70
P7	20	2	707.61

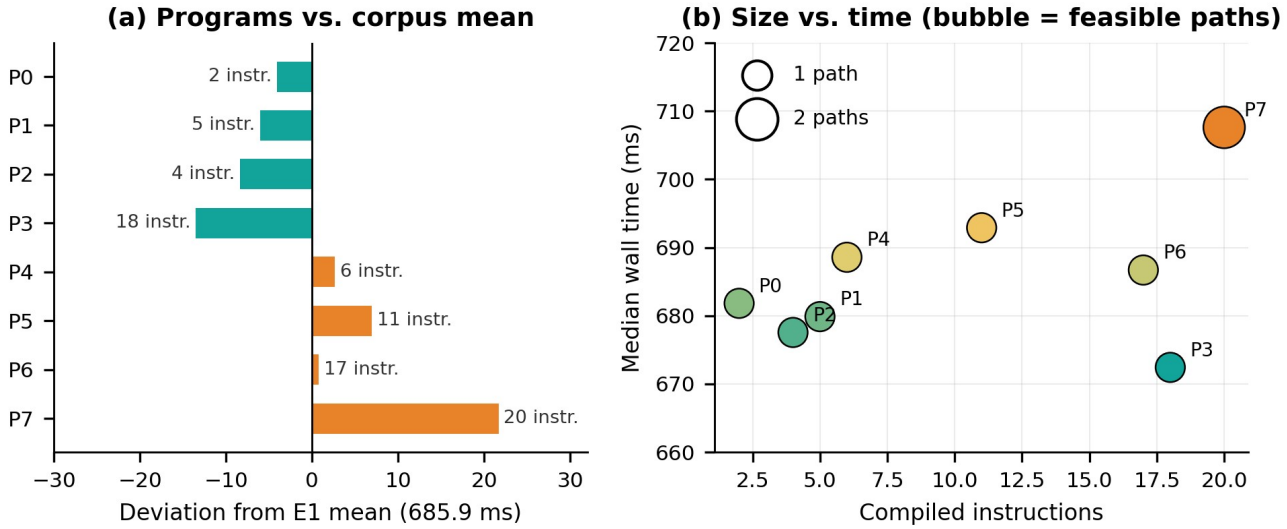


Figure 2. E1 results. (a) Deviation of each program's median from the E1 mean, annotated with its instruction count. (b) Compiled instructions against median time; bubble size shows the number of feasible paths.

The audit also found that one source-level branch (in P2) was optimized away by Clang, that the basic-block count was a heuristic, and that the field named verifier_states held KLEE's explored-path statistic. These qualifications limit how strongly the table can be read.

4.2 E2: Controlled Symbolic Path Growth

Table VI shows that cost increases steadily with the number of feasible symbolic paths. The median rises from 746.14 ms at one path to 1,231.16 ms at 64 paths, an increase of 65.0%. The project record reports $T(P) \approx 740 + 7.6P$ ms with $R^2 > 0.999$; refitting the tabulated medians gives $732 + 7.7P$ ms ($R^2 = 0.9993$). The residuals are within ±6.4 ms at every level, so the line is a good description of the tested family; it is not a complexity result.

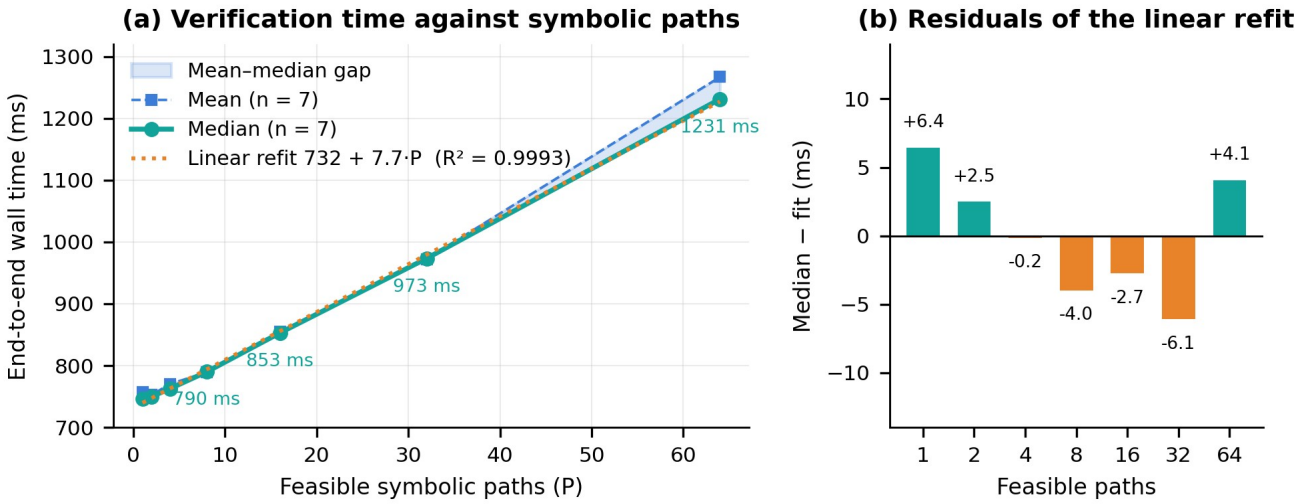


Figure 3. E2 verification cost. (a) Median and mean wall time against feasible symbolic paths, with the least-squares refit of the medians. (b) Residuals of the refit at each path count.

Table VI: E2 verification cost against feasible paths

Paths	Median (ms)	Mean (ms)	Increase (%)
1	746.14	757.28	0.0
2	749.93	752.64	0.5
4	762.77	770.22	2.2
8	789.88	789.19	5.9
16	853.02	854.79	14.3
32	973.44	972.45	30.5
64	1231.16	1266.86	65.0

Growth is sub-proportional: 64 paths cost 1.65 times the one-path median. Marginal cost is 3.8 ms for the first added path and 6.4–8.1 ms per path thereafter. At 64 paths, the mean exceeds the median by 35.7 ms, versus less than 12 ms at other levels, indicating that a few slow repetitions inflate the largest-level mean.

4.3 Fixed Overhead and Path-Dependent Cost

The refit attributes about 732 ms to fixed pipeline overhead (startup, lifting, and harness construction) and 7.7 ms × P to path-dependent cost. This component represents 8% of total cost at 8 paths, 25% at 32, and 40% at 64, adding 495 ms at 64 paths. Even one path costs about 0.75 s. This split is model-derived, not separately measured.

4.4 Comparative Workbench and Empirical Evidence

To rigorously validate hybrid performance, an independent comparative workbench executed 648 runs (504 measured runs across symbolic_only, abstract_only, and hybrid modes with 2 warmups and 7 repetitions) on the 24-program corpus. Correctness auditing across all 504 measured runs revealed 420 definitive verdicts in 100% agreement with reference classifications (zero false SAFEs and zero false VIOLATIONs), alongside 84 conservative UNKNOWN abstentions in abstract_only mode on non-abstractly solvable suites (Groups B and D) where fallback is disabled by design. In hybrid mode, the abstract stage successfully discharged 12 of 24 programs (50.0% discharge rate), slashing verification time by ~95% (~33 ms vs. ~705 ms). For the 12 programs requiring fallback, hybrid execution incurred a modest ~4.4% penalty (~752 ms vs. ~720 ms). Across all 24 programs, the paired median saving is

46.16% (aggregate reduction 45.71%; pooled 46.05%). However, because the balanced corpus contains exactly 12 discharged and 12 fallback programs, an exact two-sided sign test yields p = 1.0 (12 positive, 12 negative differences), and the 95% bootstrap confidence interval for median absolute savings spans zero (-31.62 ms to +670.72 ms). Reconciling this with historical Phase 5 records (336 measured runs, reporting 45.53% paired median and 44.93% aggregate savings) confirms consistent performance across independent campaign executions. Finally, forensic analysis of the Katran evaluation confirmed that Meta's bpf_balancer successfully compiled to a 134 KB ELF object (katran-upstream.o) and the abstract analyzer safely returned UNKNOWN. However, KRAKENGUARD's bpflifter_cli crashed with an LLVM GEP opaque pointer assertion failure (Instructions.h:960), resulting in zero verdicts and status COMPILED BUT LIFTING FAILED.

4.5 Break-Even Analysis (Model-Based)

Workbench measurements directly quantified the empirical abstract-analysis cost: mean A = 37.6 ms and median A = 33.3 ms (ranging from 31.8 ms to 49.3 ms), compared to symbolic costs S of 700–1230 ms. At median A ≈ 33.3 ms and S ≈ 705 ms, the empirical break-even discharge threshold d* = A/S is only 4.7% (and 2.7% at 64 paths). Because the tested prototype achieves a 50.0% discharge rate on the 24-program corpus, it operates substantially above the break-even threshold. For discharged programs, observed net savings reach ~672 ms per invocation, confirming that front-end abstract screening pays off dramatically when policies can be statically decided.

5. Discussion

RQ1 (program features). In this micro-corpus, instruction count does not explain end-to-end cost: a ten-fold range in instructions moved median time by only 5%. This supports treating fixed pipeline overhead and feature-dependent symbolic behavior as separate dimensions. It does not show that size is irrelevant in general, and the compiler and instrumentation qualifications apply.

RQ2 (path scaling). Cost rises monotonically and almost linearly with feasible paths, from 746 ms to 1,231 ms, at roughly 7.7 ms per path. Because about 730 ms is fixed, relative growth is small at low path counts and large only beyond 16 paths. This motivates selective analysis within the tested range, but it does not show that every workload behaves this way.

RQ3 (selective discharge). The conservative three-valued analyzer discharged 50.0% of the corpus (12 of 24 programs) with zero false SAFE and zero false VIOLATION results across 420 definitive checked executions. All 84 UNKNOWN verdicts correctly refrained from unsound assertions and routed safely to symbolic analysis in hybrid mode. While these results empirically confirm fail-closed safety on the 24-program suite, this empirical agreement represents experimental validation against a frozen symbolic oracle rather than a mechanized formal soundness proof.

RQ4 (when the hybrid pays off). The empirical hybrid workflow exhibits bimodal performance: discharged programs achieve ~95% wall-time reduction (~33 ms vs. ~705 ms), while unresolved programs incur an abstract-overhead penalty of ~4.4% (~752 ms vs. ~720 ms). On a corpus with 50% abstract solvability, the median paired reduction is 46.16%. However, because the test suite is evenly split between fast-path and fallback programs, the sign test yields p = 1.0 and the 95% confidence interval spans zero (-31.62 ms to +670.72 ms). Thus, hybrid verification is an engineering optimization whose net benefit depends directly on the proportion of statically decidable programs in the target workload. Crucially, the Katran evaluation demonstrates that front-end bytecode lifting robustness, rather than verification solver performance, is the primary operational barrier for real-world deployments.

Compared with work on verifier precision, testing, virtualization, and scheduling [3]–[7], this study isolates fine-grained policy-verification cost. In the tested regime, fixed overhead and path-dependent cost dominate, suggesting that avoiding unnecessary invocations may matter more than speeding up each invocation.

6. Conclusion

This paper presented a conservative hybrid architecture for fine-grained eBPF policy verification, placing a fast three-valued abstract interpreter ahead of a symbolic verifier. Controlled baselines demonstrate that instruction count does not dictate execution time (E1), whereas verification cost scales near-linearly with feasible execution paths over ~730 ms of fixed pipeline overhead (E2). Comparative workbench evaluation across 504 measured runs confirmed 50.0% selective discharge and a 46.16% paired median wall-time reduction with zero false acceptances or rejections. However, the sign test ($p = 1.0$) and confidence interval spanning zero establish that net savings depend heavily on workload composition. Finally, the Katran case study revealed that production adoption is constrained by LLVM bytecode lifter compatibility rather than verification theory.

7. Recommendations and Implications

I. **Practical:** analyze the compiled object, not the source; record hashes, compiler flags and policy identity; keep UNKNOWN routed to the symbolic verifier.

II. **Security and policy:** a positive result in the modeled program-policy setting is not a proof of tenant isolation; keep the kernel verifier, permissions, namespaces and cgroups in place.

III. **Managerial:** a verification gateway needs versioned policy semantics, fallback resource limits, timeout handling and authenticated provenance before production use.

IV. **Academic:** report discharge rate, abstract cost and fallback cost together, publish raw rows with aggregation scripts, and label planned or unrecoverable results as such.

V. **Future research:** harden eBPF bytecode lifting tools against modern LLVM opaque pointers; formalize mechanized soundness proofs for the abstract transfer functions; extend evaluation beyond synthetic XDP programs to multi-tenant production suites; and explore adaptive refinement of UNKNOWN states.

8. Limitations of the Study

The corpus contains 24 synthetic, policy-directed XDP programs and may not represent production workloads with complex packet parsers, bounded loops, tail calls, or multi-map coordination. Correctness is empirically evaluated against a frozen KRAKENGUARD symbolic baseline, not a formal mechanized proof. Because the 24-program corpus has an artificial 50% split of abstractly decidable cases, the overall speedup is not statistically significant ($p = 1.0$, CI spanning zero). Path scaling was evaluated across 1–64 paths in E2 but 1–32 in E5. Real-world evaluation on Katran was bounded by an LLVM assertion crash in the bytecode lifter (status: COMPILED BUT LIFTING FAILED), preventing end-to-end verification timing.

Acknowledgment. The author thanks the maintainers and authors of the open-source eBPF analysis infrastructure used as experimental baselines.

References

[1] B. Gbadamosi, L. Leonardi, T. Pulls, T. Høiland-Jørgensen, S. Ferlin-Reiter, S. Sorce, and A. Brunström, “The eBPF runtime in the Linux kernel,” arXiv preprint arXiv:2410.00026, 2024.

[2] J. Patel, L. G. Buhl-Nielsen, A. Ghosn, and M. Kogias, “KRAKENGUARD: Towards fine-grained eBPF isolation,” in Proc. 23rd USENIX Symp. Networked Syst. Design Implement. (NSDI), 2026.

[3] H. Sun and Z. Su, “Prove it to the kernel: Precise extension analysis via proof-guided abstraction refinement,” in Proc. 31st ACM Symp. Oper. Syst. Princ. (SOSP), 2025, doi: 10.1145/3731569.3764796.

[4] H. Sun and Z. Su, “Approximation enforced execution of untrusted Linux kernel extensions,” in Proc. USENIX Security Symp., 2025.

[5] T. Lyu, K. K. Dwivedi, T. Bourgeat, M. Payer, M. Xu, and S. Kashyap, “eBPF misbehavior detection: Fuzzing with a specification-based oracle,” in Proc. 31st ACM Symp. Oper. Syst. Princ. (SOSP), 2025, doi: 10.1145/3731569.3764797.

[6] J. Zhang, X. Song, D. Du, Y. Xia, B. Zang, and H. Chen, “Virtualizing eBPF with late-binding,” in Proc. 20th USENIX Symp. Oper. Syst. Design Implement. (OSDI), 2026.

[7] J. Carin, B. Holmes, W. Wang, A. Bhardwaj, and M. Ghobadi, “PeeR: First-class scheduling for latency-critical eBPF applications,” in Proc. 20th USENIX Symp. Oper. Syst. Design Implement. (OSDI), 2026.

[8] Linux Kernel Maintainers, “BPF system call documentation,” Linux Kernel Documentation, 2024–2026. [Online]. Available: https://docs.kernel.org/userspace-api/ebpf/syscall.html

[9] National Security Agency, “SeaBee: Security enhanced architecture for eBPF,” open-source project, 2025–2026. [Online]. Available: https://github.com/NationalSecurityAgency/seabee

[10] Linux Kernel Documentation, “BPF documentation,” 2026. [Online]. Available: https://docs.kernel.org/bpf/

[11] D. Thaler, “BPF instruction set architecture (ISA),” RFC 9669, 2024. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9669.html

[12] P. Cousot and R. Cousot, “Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints,” in Proc. 4th ACM SIGACT-SIGPLAN Symp. Princ. Program. Lang. (POPL), 1977, pp. 238–252.

[13] C. Cadar, D. Dunbar, and D. Engler, “KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs,” in Proc. 8th USENIX Symp. Oper. Syst. Design Implement. (OSDI), 2008.

[14] W. Findlay, A. Somayaji, and D. Barrera, “bpfbox: Simple precise process confinement with eBPF,” in Proc. Cloud Comput. Secur. Workshop (CCSW), 2020, pp. 107–120.

[15] W. Findlay, D. Barrera, and A. Somayaji, “BPFContain: Fixing the soft underbelly of container security,” arXiv preprint arXiv:2102.06972, 2021.

[16] Y. He, R. Guo, Y. Xing, X. Che, K. Sun, Z. Liu, K. Xu, and Q. Li, “Cross container attacks: The bewildered eBPF on clouds,” in Proc. 32nd USENIX Security Symp., 2023, pp. 5971–5988.